

Rabin-Karp Algorithm Example:

Problem Statement:

Given a text "abracadabra" and a pattern "abra", use the Rabin-Karp algorithm to find all occurrences of the pattern in the text.

Rabin-Karp Algorithm Steps:

1. Initialize Variables:

- Let n be the length of the text.
- Let m be the length of the pattern.
- Compute the hash values for the pattern and the first window of text.

2. Hash Function:

- A simple hash function can be used, like polynomial rolling hash.
- Compute hash value $H(s)$ for a string s as:

$$H(s) = s[0] \cdot p^{m-1} + s[1] \cdot p^{m-2} + \dots + s[m-1] \cdot p^0 \mod q$$

where p is a prime number (chosen as 31 or 53), and q is a large prime modulus.

3. Sliding the Pattern Over Text:

- Slide the pattern over text one by one and compare the hash values.
- If the hash values match, perform character-by-character comparison to confirm the match.

Example Execution:

- **Text:** "abracadabra"
- **Pattern:** "abra"
- **Pattern Length (m):** 4

- **Text Length (n):** 11
- **Prime Number (p):** 31
- **Modulus (q):** 101

Detailed Steps:

1. Compute Initial Hash Values:

- Compute hash of the pattern "abra":

$$H(\text{pattern}) = (97 \cdot 31^3 + 98 \cdot 31^2 + 114 \cdot 31^1 + 97 \cdot 31^0) \mod 101 = 4$$

- Compute hash of the first window of text "abra":

$$H(\text{window}) = (97 \cdot 31^3 + 98 \cdot 31^2 + 114 \cdot 31^1 + 97 \cdot 31^0) \mod 101 = 4$$

2. Compare and Slide:

- Compare the hash values:

$$H(\text{pattern}) = 4 \text{ and } H(\text{window}) = 4.$$

- Since hash values match, compare characters:
 - "abra" matches "abra" at index 0.

3. Slide the Window and Recompute Hash:

- Slide window to the right and compute hash for "brac":

$$H(\text{new window}) = ((H(\text{previous window}) - 97 \cdot 31^3) \cdot 31 + 99) \mod 101 = 30$$

- Compare $H(\text{pattern}) = 4$ with $H(\text{new window}) = 30$. No match.

4. Continue Sliding and Checking:

- Repeat the process for subsequent windows:
 - "raca" -> Hash = 55, no match.
 - "acad" -> Hash = 96, no match.
 - "cada" -> Hash = 3, no match.

- "adab" -> Hash = 34, no match.
- "dabr" -> Hash = 7, no match.
- "abra" -> Hash = 4, match found at index 7.

5. Pattern Occurrences:

- The pattern "abra" is found at positions 0 and 7.

CODE

```
code.py X
C:\Users\Admin\Desktop>Bhanvanu>Sem4>DAA>Lab Report> code.py > ...
1  import matplotlib.pyplot as plt
2
3  def rabin_karp_search_with_visualization(text, pattern):
4      n = len(text)
5      m = len(pattern)
6      p = 31 # Prime number for hash computation
7      q = 101 # Large prime modulus
8      p_m = pow(p, m-1, q) # p^(m-1) % q
9
10     pattern_hash = 0
11     text_hash = 0
12
13     for i in range(m):
14         pattern_hash = (pattern_hash * p + ord(pattern[i])) % q
15         text_hash = (text_hash * p + ord(text[i])) % q
16
17     matches = []
18     hash_values = []
19
20     for i in range(n - m + 1):
21         hash_values.append((i, text_hash))
22         if pattern_hash == text_hash:
23             if text[i:i + m] == pattern:
24                 matches.append(i)
25
26         if i < n - m:
27             text_hash = (text_hash - ord(text[i]) * p_m) * p + ord(text[i + m])
28             text_hash = text_hash % q
29
30     return matches, hash_values
31
32 text = "abracadabra"
33 pattern = "abra"
34 matches, hash_values = rabin_karp_search_with_visualization(text, pattern)
35
36 # Print the result
37 print("Pattern found at positions:", matches)
38
39 # Plotting the hash values
40 positions, values = zip(*hash_values)
41
42 plt.figure(figsize=(10, 6))
43 plt.plot(positions, values, marker='o')
44 plt.axhline(y=sum(ord(c) for c in pattern) % 101, color='r', linestyle='--', label='Pattern Hash Value')
45 plt.title('Rabin-Karp Hash Values for Each Text Window')
46 plt.xlabel('Position in Text')
47 plt.ylabel('Hash Value')
48 plt.legend()
49 plt.xticks(positions)
50 plt.grid(True)
51 plt.show()
52
```